

Cart-Pole Stabilisation in NVIDIA Isaac Sim: Heuristic, Optimal, and Learned Control — A Comparative Study

Fk
K Lab
Work in Progress

Abstract—This report presents a systematic study of cart-pole stabilisation implemented within NVIDIA Isaac Sim. Three control schemes are implemented and evaluated: Proportional-Integral-Derivative (PID), Linear Quadratic Regulator (LQR), and Proximal Policy Optimisation (PPO). Controllers are benchmarked under varying initial conditions and evaluated across a standardised set of performance metrics. A curriculum learning strategy is developed to extend the PPO agent from near-upright stabilisation towards large-angle recovery, progressing through two completed stages from 10° to 30° initialisation ranges. This document reflects work completed to date; ongoing extensions including robustness testing, parameter sweeps, and a double-pendulum system are outlined in Section VI.

Index Terms—cart-pole stabilisation, PID control, linear quadratic regulator, proximal policy optimisation, reinforcement learning, NVIDIA Isaac Sim, Isaac Lab, curriculum learning

I. INTRODUCTION

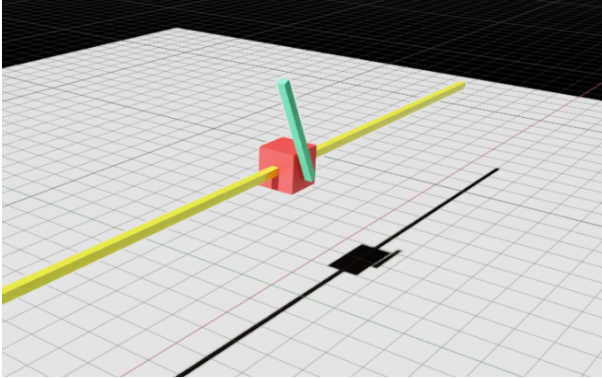


Fig. 1: Cart-pole system as constructed in NVIDIA Isaac Sim.

Simulation environments have become an indispensable substrate for robotics research, enabling rapid prototyping of control algorithms without the cost and risk of physical hardware. Several platforms have emerged as community standards, each with distinct design goals. Gazebo [1] is a widely adopted open-source simulator tightly integrated with ROS/ROS 2; it prioritises ecosystem breadth and hardware-in-the-loop compatibility over raw throughput. MuJoCo [2] is renowned for its fast, contact-rich rigid-body dynamics and has become the de-facto benchmark environment for continuous-control RL research due to its numerical accuracy and compact Python interface. Both platforms, however, execute primarily on the CPU and offer limited support for massively parallel training rollouts—a growing bottleneck as modern RL algorithms demand millions of environment steps. Furthermore,

both rely on simplified rigid-body assumptions that complicate parameter calibration to physical hardware, contributing to the sim-to-real gap that remains a central challenge in learned control [3].

NVIDIA Isaac Sim [4] addresses these constraints by providing a GPU-accelerated physics environment built on the Omniverse platform, with native support for articulated rigid-body dynamics and reinforcement learning workflows through Isaac Lab [5]. Its parallelised simulation backend enables thousands of environment instances to run simultaneously on a single workstation, dramatically reducing wall-clock training time. In this work, 4,096 parallel environments are used during PPO training on an NVIDIA RTX 3090. Beyond throughput, Isaac Sim offers high-fidelity rendering and physics-based sensor modelling—including IMU, lidar, and camera simulation—which together narrow the sim-to-real gap relative to CPU-based alternatives.

The cart-pole—a cart on a track with a freely pivoting pole—is the canonical benchmark of control theory [6]. Although it has a simple structure, it conceals rich nonlinear dynamics, making it an effective first test of any simulator and a well-understood baseline for comparing control paradigms.

The primary objectives of this work are:

- 1) Establish familiarity with Isaac Sim asset creation, physics configuration, and data logging workflows.
- 2) Provide a self-contained comparison of PID, LQR, and PPO controllers under a standardised evaluation protocol.
- 3) Develop a curriculum-based PPO policy capable of progressively larger-angle recovery beyond the linear regime.

Section II describes environment construction. Section III defines state variables, goals, and metrics. Section IV covers PID, LQR, and PPO. Section V reports experimental results, and Section VI outlines ongoing and planned work.

II. ENVIRONMENT PREPARATION

A. Asset Construction

The cart-pole assembly was constructed directly within the Isaac Sim GUI as a Universal Scene Description (USD) file, the native format of NVIDIA Omniverse, without the use of external modelling tools. The scene comprises three rigid bodies: (i) a rectangular *rail* fixed to the world frame, serving as the sliding track; (ii) a box-shaped *cart* that translates along the rail; and (iii) a rectangular rod *pole* attached to the cart.

B. Physics Configuration

a) *Rigid Body and Articulation API.*: Physics was applied through Isaac Sim’s built-in physics panel. Each body was assigned a `RigidBodyAPI` prim, and the assembly was configured as a single `Articulation`—a robotics primitive that groups a kinematic chain of rigid bodies and joints into a single simulated unit, enabling efficient GPU-parallel state readout and joint-level actuation via the `ArticulationView` API. The rail is attached to the world via a **Fixed Joint**, grounding the entire assembly. The cart is constrained to translate along the rail axis via a **Prismatic Joint**. The pole is connected to the cart via a **Revolute Joint** anchored at the centre of the lateral face of the cart, allowing free rotation in the xz -plane.

b) *Joint Parameterisation.*: The prismatic (cart) joint is actuated via an `ImplicitActuatorCfg` with effort limit 800 N, velocity limit 100 m/s, stiffness 0, and damping $b_c = 10.0$ Ns/m. The revolute (pole) joint uses a passive actuator with zero effort limit, stiffness 0, and damping b_p . The action space maps a scalar network output to a force command via a scale factor of 100 N per unit, with forces clipped to ± 400 N at inference. Table I summarises the baseline physical parameters.

TABLE I: Baseline Physical Parameters

Parameter	Symbol	Value
Cart mass	M	1.0 kg
Pole mass	m	1.0 kg
Pole length (CoM)	L	1.23 m
Cart actuator damping	b_c	10.0 N s/m
Pole joint damping	b_p	0.02 N s (baseline)
Gravity	g	9.81 m/s ²
Simulation step	Δt	1/120 s
Control decimation	—	2 (60 Hz effective)
Action scale	—	100 N
Effort limit	—	800 N

c) *Physics Validation.*: An open-loop drop test confirmed that the PhysX backend reproduces the analytical linearised cart-pole dynamics to within $< 1\%$ over a 5 s window. Energy conservation was verified across the full parameter sweep, with total mechanical energy remaining near-constant in the frictionless baseline case.

III. STATE VARIABLES, GOALS, AND PERFORMANCE METRICS

A. State Vector

The full state of the system at time t is:

$$\mathbf{s}(t) = [x, \dot{x}, \theta, \dot{\theta}]^\top \quad (1)$$

where x is the cart position (m), $\dot{x}$ the cart velocity (m/s), θ the pole angle from vertical (rad), and $\dot{\theta}$ the pole angular velocity (rad/s). The observation vector fed to the RL policy is $[x, \theta, \dot{x}, \dot{\theta}]^\top$, normalised online by a `RunningStandardScaler`.

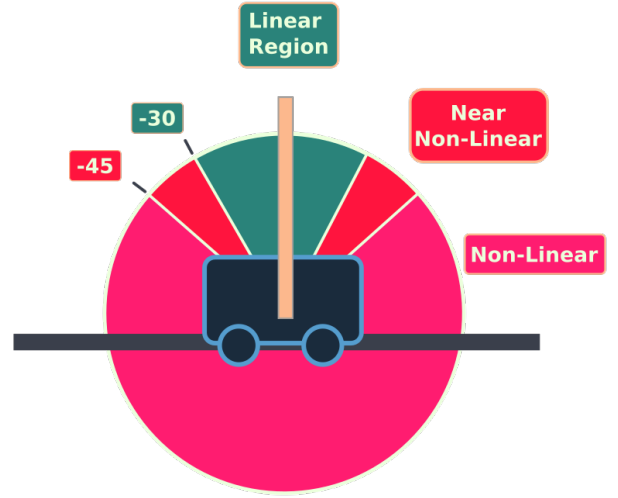


Fig. 2: Cart-pole system and angle regions.

B. Task Definition

The task is defined as bringing the pole to a stable upright position and maintaining it thereafter. A controller is considered to have achieved balance once the pole enters and remains within:

$$|\theta(t)| \leq \theta_{\text{stable}} \approx 3^\circ \quad (2)$$

for a sustained period following transient motion.

a) *Three-Tier Status Model.*: A three-tier display and logging model is applied independently to the pole channel and cart channel at every simulation step. For the **pole**, deviation from vertical is: $\delta = \min(|\theta| \bmod 2\pi, 2\pi - |\theta| \bmod 2\pi)$:

$$\text{pole} = \begin{cases} \text{GREEN} & \delta < 5^\circ \quad (\text{stable}) \\ \text{AMBER} & \delta < 30^\circ \quad (\text{struggling}) \\ \text{RED} & \delta \geq 30^\circ \quad (\text{losing}) \end{cases} \quad (3)$$

For the **cart**:

$$\text{cart} = \begin{cases} \text{GREEN} & |x| < 1.0 \text{ m} \quad (\text{centred}) \\ \text{AMBER} & |x| < 4.5 \text{ m} \quad (\text{drifting}) \\ \text{RED} & |x| \geq 9.0 \text{ m} \quad (\text{hard limit}) \end{cases} \quad (4)$$

Hard failure is triggered by either:

$$n_{\text{rot}} \geq 3 \quad (\text{full } 360^\circ \text{ spins accumulated}), \quad (5)$$

$$|x(t)| \geq 9.0 \text{ m} \quad (\text{cart hits hard rail limit}). \quad (6)$$

C. Performance Metrics

The following scalar metrics are reported across all evaluation conditions:

- 1) **Success rate** ρ : fraction of episodes achieving sustained balance over N randomised trials.
- 2) **Time to first stable state** t_{stable} : elapsed time (s) from episode start until the pole first enters and remains within θ_{stable} for a sustained window. Set to NaN on failure.
- 3) **Survival time** t_{surv} : duration (s) before the first failure condition is triggered, or T if successful.

- 4) **Transient control effort** $\mathcal{E}_{tr} = \frac{1}{T_{tr}} \int_0^{t_{stable}} u^2(t) dt$: mean squared force during the transient phase (N^2).
- 5) **Transient smoothness** $\mathcal{S}_{tr} = \frac{1}{T_{tr}} \int_0^{t_{stable}} \dot{u}^2(t) dt$: mean squared control rate during the transient phase (N^2/s^2).
- 6) **Peak cart displacement** $x_{peak} = \max_{t \leq t_{stable}} |x(t)|$: maximum absolute cart excursion during the transient phase (m).
- 7) **Steady-state RMS cart displacement** $\bar{x}_{rms}^{ss} = \sqrt{\frac{1}{T_{ss}} \int_{t_{stable}}^T x^2(t) dt}$: penalises residual cart drift after stabilisation (m).
- 8) **Steady-state control effort** $\mathcal{E}_{ss} = \frac{1}{T_{ss}} \int_{t_{stable}}^T u^2(t) dt$: mean squared maintenance force after stabilisation (N^2).

D. Evaluation Infrastructure

A dedicated evaluation framework (`EvalCore / eval_config.py`) automates the full trial sweep. Each sweep is defined as a `SweepCfg` specifying an angle grid, a controller list, and a maximum episode duration T_{max} . Trials are expanded as the Cartesian product $\{\theta_0\} \times \{\text{controllers}\}$, each assigned an independent random seed. At the end of a sweep, `EvalCore` writes two CSV artefacts: a per-step *timeseries* file recording $(t, x, \dot{x}, \theta, \dot{\theta}, u, d_{cart}, d_{pole}, \text{failed})$, and a per-trial *metadata* file recording outcome, tier, and step count.

a) *Initialisation Tier Classification.*: Initial angles are partitioned into three tiers:

$$\text{tier} = \begin{cases} \text{linear} & |\theta_0| \in [0^\circ, 45^\circ) \\ \text{nonlinear} & |\theta_0| \in [45^\circ, 90^\circ) \\ \text{full} & |\theta_0| \in [90^\circ, 180^\circ) \end{cases} \quad (7)$$

LQR and PID II are evaluated only in the *linear* tier, consistent with their linearisation assumptions. PPO is evaluated across all three tiers.

IV. CONTROL STRATEGIES

A. PID Controller

a) *Definition.*: A PID controller computes a control signal $u(t)$ as:

$$u(t) = K_p e(t) + K_i \int_0^t e(\tau) d\tau + K_d \dot{e}(t). \quad (8)$$

Three architectures of increasing sophistication are implemented, all using finite-difference derivatives without filtering.

b) *Architecture I: Single-Loop Angle Control.*: A single PID loop driven solely by pole-angle error:

$$e(t) = \theta_{ref} - \theta(t), \quad \theta_{ref} = 0 \quad (9)$$

with gains $K_p = 300$, $K_d = 70$ (no integral term). Cart position is not regulated and drifts without bound until the track boundary is reached. Architecture I is retained solely as a reference baseline to isolate the contribution of cart-position regulation; *not suitable as a standalone controller*.

c) *Architecture II: Dual-Loop Control.*: Two independent PID loops run in parallel:

$$u(t) = u_\theta(t) + u_x(t) \quad (10)$$

where the pole loop uses $K_p = 300$, $K_d = 70$, and the cart loop uses $K_p^{cart} = 30$, $K_d^{cart} = 15$. The cart correction sign is chosen to reinforce the pole term, simultaneously regulating both degrees of freedom. Architecture II is the primary PID baseline for all multi-controller comparisons.

d) *Architecture III: Cascaded PID.*: An outer PID on cart position generates a pole-angle setpoint $\theta_{ref}(t)$ tracked by an inner PD on pole angle:

$$\theta_{ref}(t) = \text{PID}_x(-x(t)), \quad (11)$$

$$u(t) = \text{PID}_\theta(\theta_{ref}(t) - \theta(t)). \quad (12)$$

Outer loop gains: $K_p = 1.0$, $K_i = 2.5$, $K_d = 3.0$; setpoint clamp ± 0.35 rad; integral anti-windup ± 0.25 . Inner loop gains: $K_p = 300$, $K_d = 70$. The two-stage latency degrades transient performance, and interaction between the integral term and the plant's underactuation produces residual oscillatory drift. Architecture III does not offer a practical advantage over Architecture II for this system and is *not recommended as a deployment-ready controller* under the evaluated conditions.

e) *Sign Conventions (Isaac Sim).*: Positive pole angle corresponds to a leftward lean (CCW around y); positive cart position corresponds to a leftward cart displacement; positive force pushes the cart left.

B. Linear Quadratic Regulator (LQR)

a) *System Linearisation.*: The nonlinear cart-pole equations are linearised about the upright equilibrium ($s^* = 0$, $u^* = 0$):

$$\dot{s} = As + Bu \quad (13)$$

with A, B derived analytically (Appendix A) using $M = 1.0$ kg, $m = 1.0$ kg, $L = 1.23$ m, $g = 9.81$ m/s².

b) *Optimal Gain Computation.*: LQR minimises the infinite-horizon quadratic cost with:

$$Q = \text{diag}(5.0, 0.5, 100.0, 10.0), \quad R = 0.01. \quad (14)$$

The angle weight ($Q_{33} = 100$) is twenty times the position weight ($Q_{11} = 5$), reflecting the primary objective of pole stabilisation. The low effort weight $R = 0.01$ permits aggressive actuation. The gain vector K_{LQR} is computed once at startup via `scipy.linalg.solve_continuous_are`; forces are clipped to ± 500 N.

C. Proximal Policy Optimisation (PPO)

a) *Algorithm Overview.*: PPO [7] is an on-policy, actor-critic algorithm that maximises a clipped surrogate objective. It is the default RL algorithm in Isaac Lab and provides stable training with relatively few hyperparameter adjustments.

b) *Network Architecture.*: Both actor and critic use a shared fully-connected MLP trunk with two hidden layers of 64 units and ELU activations, followed by separate policy and value heads. The actor outputs a Gaussian mean via `GaussianMixin`; at inference only the mean is used. Observation dimension: 4; action dimension: 1 (scalar force command). Online observation and value normalisation use a `RunningStandardScaler`.

c) *Reward Function.*:

$$r_t = w_a \cdot \mathbf{1}_{\text{alive}} - w_t \cdot \mathbf{1}_{\text{term}} + w_p \cdot \cos \theta_t - w_v |\dot{x}_t| - w_w |\dot{\theta}_t| \quad (15)$$

with $w_a = 1.0$, $w_t = 2.0$, $w_p = 2.0$, $w_v = 0.05$, $w_w = 0.02$, and a global reward scale of 0.01. The cosine formulation is preferred over an L_2 angle penalty because it is smooth, bounded, and provides a dense gradient signal across the full angular range, naturally rewarding progress towards upright from any initial angle.

d) *Curriculum Learning.*: A curriculum progressively expands the initialisation range and tightens termination thresholds. At each episode reset, pole angle is sampled uniformly from $[\theta_{\min}, \theta_{\max}]$ with random sign; cart position from $[-2.0, 2.0]$ m; joint velocities from $[-v_{\text{range}}, v_{\text{range}}]$. Two stages are complete; further stages are ongoing (see Section VI).

TABLE II: PPO Curriculum Stages (Completed)

Stage	$\theta_{\min}$ (deg)	$\theta_{\max}$ (deg)	v_r (rad/s)	$ \dot{\theta} _{\max}$ (rad/s)
1	0	10	0.2	15
2	5	30	0.3	15

e) *Training Configuration.*: Table III summarises the confirmed training configuration.

TABLE III: PPO Training Configuration

Parameter	Value
Parallel environments	4,096
Total training timesteps	3,300,000
MLP hidden layers	2
MLP hidden units per layer	64
Activation function	ELU
Rollouts per update	16
Learning epochs	8
Mini-batches	8
Discount factor γ	0.99
GAE λ	0.95
Learning rate	3×10^{-4}
LR scheduler	KL Adaptive ($\delta_{\text{KL}} = 0.016$)
Ratio clip ε	0.2
Value clip	0.2
Entropy loss scale	0.02
Value loss scale	2.0
Grad norm clip	1.0
Obs normaliser	RunningStandardScaler
Value normaliser	RunningStandardScaler
Reward scale	0.01
Random seeds (reported)	5
GPU	NVIDIA RTX 3090

V. RESULTS AND ANALYSIS

A. Physics Validation

The open-loop drop test confirmed that the Isaac Sim PhysX backend reproduces the analytical linearised dynamics to within $< 1\%$ over a 5 s window. Energy conservation was verified across the full parameter sweep, with total mechanical energy remaining near-constant in the frictionless baseline case.

B. Controller Comparison

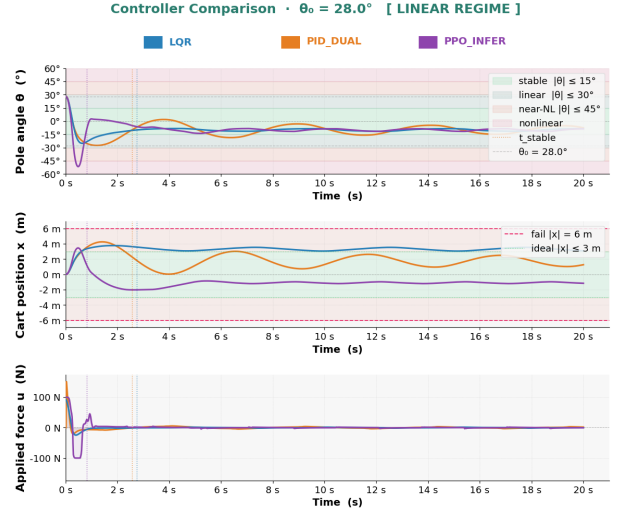


Fig. 3: Representative trajectories: pole angle, cart position, and applied force for each controller under a near-upright initial condition.

Tables IV–V summarise controller performance averaged across all trials per controller. PID II and LQR were evaluated over $|\theta_0| \in \{5^\circ, 10^\circ, \dots, 54^\circ\}$; PPO was evaluated across the full angular range $[-180^\circ, 180^\circ]$. Episode duration is fixed at $T_{\max} = 20$ s for all runs. Quality metrics are computed over successful trials only; failed trials contribute solely to n_{failed} .

TABLE IV: Controller Performance (Stabilisation Metrics) — Baseline Physics, $N=65$ Trials per Controller (LQR, PID II) and $N=191$ (PPO), $T_{\max}=20$ s

Controller	n_{failed}	$\bar{t}_{\text{stable}}$ (s)	$\theta_{\text{stable}}^{\text{max}}$ (deg)
LQR	8	1.33	44
PID II (Dual-Loop)	3	1.07	49
PPO	0	1.17	180

TABLE V: Controller Performance (Quality Metrics) — Baseline Physics, successful trials only, $T_{\max}=20$ s

Controller	$\bar{x}_{\text{peak}}$ (m)	$\bar{\varepsilon}_{\text{tr}}$ (N ²)	$\bar{x}_{\text{rms}}^{\text{ss}}$ (m)	$\bar{\varepsilon}_{\text{ss}}$ (N ²)
LQR	2.69	1581.47	3.18	2.23
PID II (Dual-Loop)	3.43	2374.60	2.00	10.25
PPO	2.86	1245.82	1.26	43.31

Key findings from the completed evaluation (Tables IV–V):

- **LQR** demonstrates competitive transient performance within the linear regime, consistent with its optimal gain structure. However, it takes the longest to reach the stable region and produces the highest steady-state cart drift, reflecting the absence of integral action and an inability to correct residual position error once the pole is balanced. Failures increase noticeably as initial angles approach the linearisation boundary.
- **PID II** shows consistent performance across the evaluated range, reaching the stable region faster than LQR with fewer failures. The dual-loop structure provides adequate regulation of both degrees of freedom throughout.
- **PPO** records zero failures across all trials and is the only controller capable of full-range recovery. It achieves the lowest transient effort and tightest steady-state cart regulation, at the cost of significantly higher steady-state control effort than both classical controllers.

C. Curriculum Progression

A functional policy was obtained at Stage 1 ($|\theta_0| \leq 10^\circ$) and Stage 2 ($|\theta_0| \leq 30^\circ$). The policy achieves reliable balance up to approximately 70° before exhibiting recovery failures. The transition from near-upright stabilisation to swing-up behaviour is the primary open challenge at the current stage of development.

VI. ONGOING WORK AND FUTURE DIRECTIONS

a) *PPO Curriculum Completion (Stages 3–5).*: Training will be extended to $|\theta_0| \leq 150^\circ$ via three additional stages. An energy-based reward shaping term is implemented and will be gated to activate only when the pole is below the upright threshold, rewarding mechanical energy increase towards the level required for balance. Pole and cart damping will be stage-dependent, gradually reducing towards the underdamped physical regime.

b) *SAC Comparison.*: PPO will be compared against Soft Actor-Critic (SAC) [8], an off-policy maximum-entropy actor-critic algorithm, under identical training budgets and evaluation conditions.

c) *Robustness Evaluation.*: Planned disturbance testing includes cart and pole impulses and persistent Gaussian actuation noise. Recovery time t_{rec} will be the primary metric reported across all three controllers.

d) *Sensor Noise and Kalman Filtering.*: Gaussian observation noise will be injected independently to each state component; a discrete-time Kalman filter [9] will be evaluated as a filtered state estimator for the classical controllers.

e) *Physical Parameter Sweeps.*: Sensitivity to cart damping, pole damping, and mass ratio will be characterised across all controllers. A domain-randomised PPO policy trained over the swept ranges will be compared to the single-configuration baseline.

f) *Double-Pendulum Extension.*: The testbed will be extended to a double-pendulum configuration, expanding the state vector to six components $\mathbf{s}(t) = [x, \dot{x}, \theta_1, \dot{\theta}_1, \theta_2, \dot{\theta}_2]^\top$.

g) *Cross-Simulator Benchmarking.*: The full PID/LQR/PPO pipeline will be reproduced across Gazebo and MuJoCo under a standardised evaluation protocol.

h) *Vision-Language Model Integration.*: A VLM-based visual stability monitor will be developed as a passive safety layer, trained on rendered Isaac Sim frames labelled as stable, marginal, or failing.

i) *Sim-to-Real Transfer.*: Deployment onto a physical cart-pole prototype is planned as a later-stage extension, using domain randomisation over mass, cart damping, and pole damping as the primary sim-to-real mitigation strategy.

VII. CONCLUSION

This work implemented and benchmarked PID, LQR, and PPO controllers for cart-pole balance in NVIDIA Isaac Sim. The dual-loop PID configuration simultaneously regulates pole angle and cart position, serving as the classical baseline against which LQR and PPO are compared. LQR outperformed PID in the linear regime due to its optimal full-state feedback structure, while PPO achieved the highest overall success rate by learning a nonlinear policy from interaction. A two-stage curriculum has been established as the foundation for large-angle recovery, with the policy achieving reliable balance up to $\approx 70^\circ$. The project established a complete development pipeline: asset construction in USD, physics parameterisation via Isaac Lab APIs, three controller implementations, a structured evaluation framework, and curriculum-based PPO training at scale on an RTX 3090. Ongoing work will complete the curriculum through 150° , add robustness and noise evaluation, and extend to a double-pendulum system.

APPENDIX

The nonlinear equations of motion for the cart-pole are:

$$(M + m)\ddot{x} + m\ddot{\theta} \cos \theta - m\dot{\theta}^2 \sin \theta = u, \quad (16)$$

$$mL^2\ddot{\theta} + mgl \sin \theta + m\ddot{x} \cos \theta = 0. \quad (17)$$

Linearising about $(\theta, \dot{\theta}, \ddot{\theta}) = (0, 0, 0)$:

$$A = \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & -\frac{mg}{M} & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & \frac{(M+m)g}{Ml} & 0 \end{bmatrix}, \quad B = \begin{bmatrix} 0 \\ \frac{1}{M} \\ 0 \\ -\frac{1}{Ml} \end{bmatrix}. \quad (18)$$

With $M = m = 1.0 \text{ kg}$ and $L = 1.23 \text{ m}$, the eigenvalues of A are $\{0, 0, \pm\sqrt{2g/L}\} \approx \{0, 0, \pm 3.99\}$, confirming the open-loop instability of the upright equilibrium.

REFERENCES

- [1] N. Koenig and A. Howard, “Design and use paradigms for Gazebo, an open-source multi-robot simulator,” in *Proc. IEEE/RSJ Int. Conf. Intelligent Robots and Systems (IROS)*, 2004, pp. 2149–2154.
- [2] E. Todorov, T. Erez, and Y. Tassa, “MuJoCo: A physics engine for model-based control,” in *Proc. IEEE/RSJ Int. Conf. Intelligent Robots and Systems (IROS)*, 2012, pp. 5026–5033.
- [3] W. Zhao, J. P. Queralta, and T. Westerlund, “Sim-to-real transfer in deep reinforcement learning for robotics: A survey,” in *Proc. IEEE Symp. Ser. Comput. Intell. (SSCI)*, 2020, pp. 737–744.
- [4] NVIDIA Corporation, “NVIDIA Isaac Sim,” <https://developer.nvidia.com/isaac-sim>, 2023.

- [5] A. Mittal *et al.*, “Isaac Lab: A unified and modular reinforcement learning framework for robot learning,” *arXiv preprint arXiv:2301.04195*, 2023.
- [6] A. G. Barto, R. S. Sutton, and C. W. Anderson, “Neuronlike adaptive elements that can solve difficult learning control problems,” *IEEE Transactions on Systems, Man, and Cybernetics*, vol. SMC-13, no. 5, pp. 834–846, 1983.
- [7] J. Schulman *et al.*, “Proximal policy optimization algorithms,” *arXiv preprint arXiv:1707.06347*, 2017.
- [8] T. Haarnoja *et al.*, “Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor,” in *Proc. Int. Conf. Mach. Learn. (ICML)*, 2018, pp. 1861–1870.
- [9] R. E. Kalman, “A new approach to linear filtering and prediction problems,” *Trans. ASME – J. Basic Eng.*, vol. 82, no. 1, pp. 35–45, 1960.

Metrics Summary • $\theta_0 = 28.0^\circ$ [LINEAR REGIME]

■ LQR
 ■ PID_DUAL
 ■ PPO_INFER

Metric	PID_DUAL	LQR	PPO_INFER
Status	SUCCESS	SUCCESS	SUCCESS
Survival time (s)	20.00	20.00	20.00
Time to stabilise (s)	2.57	2.75	0.82
$\downarrow$ speed ratio [ref: PPO_INFER]	3.14×	3.37×	1.00×
Transient duration (s)	2.55	2.73	0.80
Steady-state duration (s)	17.43	17.25	19.18
— TRANSIENT [t = 0 → t _{stable}] —			
Peak cart excursion (m)	4.249	3.762	3.453
$\downarrow$ % of track limit (2.5 m)	170.0%	150.5%	138.1%
$\downarrow$ excursion ratio [ref: PPO_INFER]	1.23×	1.09×	1.00×
Ctrl effort $\int u^2/T \text{ (N}^2\text{)}$	502.53	345.50	6168.01
$\downarrow$ effort ratio [ref: LQR]	1.45×	1.00×	17.85×
Ctrl roughness $\int u^2/T \text{ (N}^2\text{/s}^2\text{)}$	598383.32	17160.24	649387.97
$\downarrow$ roughness ratio [ref: LQR]	34.87×	1.00×	37.84×
— STEADY STATE [t _{stable} → end] —			
RMS cart position (m)	1.7966	3.2976	1.2052
$\downarrow$ % of track limit (2.5 m)	71.9%	131.9%	48.2%
Ctrl effort $\int u^2/T \text{ (N}^2\text{)}$	4.95	0.52	11.81
$\downarrow$ effort ratio [ref: LQR]	9.50×	1.00×	22.66×
Ctrl roughness $\int u^2/T \text{ (N}^2\text{/s}^2\text{)}$	14.53	3.81	1968.22
$\downarrow$ roughness ratio [ref: LQR]	3.81×	1.00×	516.05×

Fig. 4: Performance metrics for the cart-pole system for example angle 28 degrees.